



PROJET DE FIN DE SEMESTRE

SecureBank : Auth System

Modélisation, Vérification Formelle et Analyse Big Data d'un Système
d'Authentification Distribué

Étudiants :

Rayan SAIL

Younes DOUICI

Mohamed TOUNKARA

Matthieu ALLOCHON

Colince FONO

Enseignante :

Mme ZAUCHE Djaouda

3 mai 2026

Table des matières

1	Architecture Globale du Système	2
1.1	Contexte et Objectifs	2
1.2	Diagramme d'Architecture des Acteurs	2
1.3	Environnement Technique	2
1.4	Définition des Protocoles	3
1.5	Machine à États du Client	3
1.6	Cycle de Vie et Machine à États	3
2	Vérification Formelle par Réseau de Pétri	4
2.1	Modélisation des États	4
2.2	Représentation Graphique du Réseau	4
2.3	Matrices de Transition (Pre et Post)	5
2.4	Analyse des Invariants et LTL	5
2.5	Analyse critique du modèle de Pétri	5
3	Analyse de Sécurité et Big Data	6
3.1	Pipeline de Données et Persistance	6
3.2	Algorithme de Détection de Brute-Force	6
3.3	Détails des Scénarios de Simulation	7
3.3.1	Scénario 1 : Authentification Nominale	7
3.3.2	Scénario 2 : Attaque Brute-force	7
3.3.3	Scénario 3 : Replay Attack	7
3.3.4	Scénario 4 : Credential Stuffing	7
4	Validation et Intégration	8
4.1	Stratégie de Test avec ScalaTest	8
4.1.1	Tests Unitaires (SecurityAnalyzerSpec)	8
4.1.2	Tests d'Intégration (SecureBankIntegrationSpec)	8
4.2	Couverture Fonctionnelle	8

Chapitre 1

Architecture Globale du Système

1.1 Contexte et Objectifs

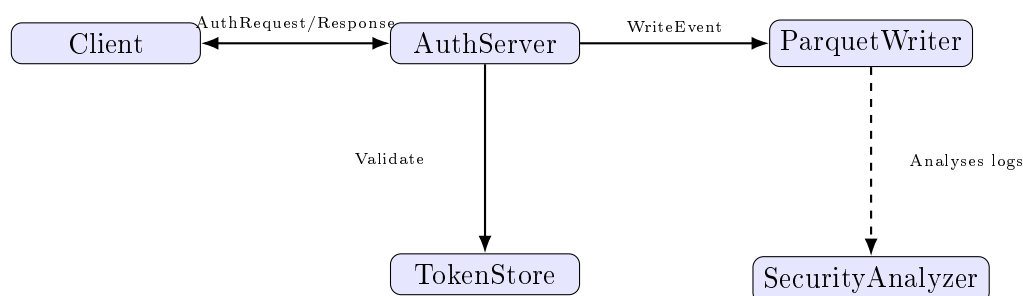
Le projet **SecureBank** vise à concevoir un système d'authentification robuste utilisant le modèle d'acteurs[cite : 6]. Contrairement aux systèmes monolithiques, SecureBank repose sur une architecture distribuée où chaque entité (Client, AuthServer, TokenStore) est isolée.

Objectifs Principaux

- Gestion sécurisée des sessions via tokens JWT.
- Résilience face aux attaques de type Brute-force.
- Vérification mathématique du comportement via Réseaux de Pétri.
- Analyse post-mortem des logs via un pipeline Apache Spark.

1.2 Diagramme d'Architecture des Acteurs

Le système est composé de 5 acteurs principaux échangeant des messages asynchrones[cite : 6].



1.3 Environnement Technique

Le projet s'appuie sur une stack logicielle moderne et robuste, garantissant la compatibilité entre le traitement distribué et l'analyse de données[cite : 6].

Composant	Version	Rôle
Scala	2.13.12	Langage de programmation principal
Akka Typed	2.6.20	Framework d'acteurs et messages[cite : 6]
Apache Spark	3.4.1	Analyse Big Data (format Parquet)
SBT	1.9.8	Gestionnaire de dépendances et build
ScalaTest	3.2.17	Tests unitaires et d'intégration

TABLE 1.1 – Stack technique du système SecureBank

1.4 Définition des Protocoles

Comme détaillé dans le fichier `Protocols.scala`[cite : 6], la communication entre les acteurs est strictement typée. Cela garantit qu'un acteur ne traite que les messages qu'il est capable de comprendre.

Listing 1.1 – Extraits des messages de l'AuthServer

```

1 // Source: Protocols.scala
2 sealed trait AuthCommand
3 final case class Authenticate(
4     user: String,
5     pass: String,
6     replyTo: ActorRef[AuthResponse]
7 ) extends AuthCommand

```

1.5 Machine à États du Client

Le comportement du client est modélisé par une machine à états finis (FSM) intégrée dans `Client.scala`.

Transitions d'états

- **Idle** → **AwaitingAuth** : Envoi des identifiants.
- **AwaitingAuth** → **Session** : Réception d'un `AuthSuccess`.
- **AwaitingAuth** → **Bloqué** : Réception de `AccountLocked` après 3 échecs.

1.6 Cycle de Vie et Machine à États

La logique du `Client.scala` est structurée comme une machine à états finis, assurant qu'aucune opération n'est effectuée hors contexte.

Transitions d'états du Client

- **Idle** : État initial, attend l'ordre de connexion[cite : 6].
- **AwaitingAuth** : Requête envoyée, attend la réponse de l' `AuthServer`.
- **Session** : Authentification réussie, accès aux ressources autorisé.
- **Bloqué** : État absorbant atteint après détection de brute-force.

Chapitre 2

Vérification Formelle par Réseau de Pétri

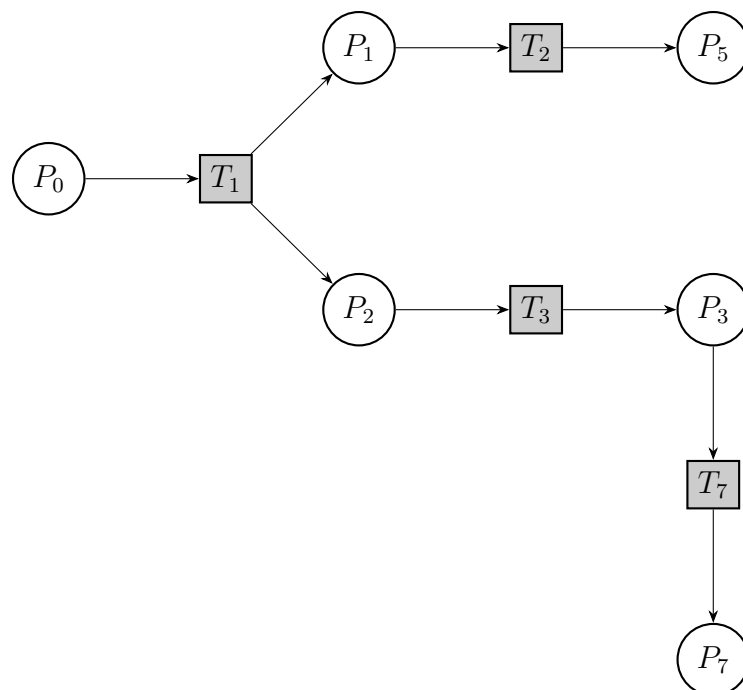
2.1 Modélisation des États

Le système a été modélisé dans `PetriNet.scala` pour prouver son absence de deadlock et la sécurité de ses transitions[cite : 6].

Places Clés du Réseau

- **P0 (Idle)** : Client en attente (1 jeton initial).
- **P3 (FailedAttempt)** : Compteur d'échecs cumulatif.
- **P7 (AccountLocked)** : Compte verrouillé définitivement après 3 jetons en P3.

2.2 Représentation Graphique du Réseau



2.3 Matrices de Transition (Pre et Post)

Les matrices d'incidence définissent l'évolution du marquage du réseau[cite : 6].

$$Pre = \begin{pmatrix} 1 & 0 & 0 & \dots \\ 0 & 1 & 0 & \dots \\ 0 & 0 & 1 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}, \quad Post = \begin{pmatrix} 0 & 1 & 0 & \dots \\ 1 & 0 & 0 & \dots \\ 0 & 1 & 1 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

2.4 Analyse des Invariants et LTL

L'analyse automatisée a permis de vérifier les propriétés suivantes[cite : 6] :

- **LTL1 (Accessibilité)** : $\Diamond valid_session$. Résultat : **Faux** (une erreur peut bloquer définitivement l'accès).
- **LTL2 (Vivacité)** : $G(P_0 \rightarrow \Diamond(P_5 \vee P_7))$. Résultat : **Vrai** (le processus se termine toujours).
- **LTL3 (Sécurité)** : $G(account_locked \rightarrow \neg valid_session)$. Résultat : **Vrai**.
- **LTL4 (Persistance)** : $G(P_7 \rightarrow GP_7)$. Résultat : **Vrai** (P7 est un état absorbant).

Cette propriété LTL3 est garantie par l'ajout de gardes dans `PetriNet.scala` :

```
1 // Gardes pour assurer l'exclusion mutuelle session/verrouillage
2 val guards = Map(
3   "T7" -> Map("P5" -> ((t: Int) => t == 0)), // Bloque le
         verrouillage si P5 active
4   "T4" -> Map("P7" -> ((t: Int) => t == 0)) // Bloque la
         session si P7 active
5 )
```

2.5 Analyse critique du modèle de Pétri

Une distinction majeure apparaît entre la vérification formelle et l'implémentation réelle :

- **Abstraction du modèle** : En Pétri scalaire, les jetons sont anonymes. Le modèle ne peut pas distinguer deux tokens différents, ce qui rend les propriétés LTL1 et LTL4 "Fausses" mathématiquement.
- **Précision logicielle** : Dans le système Akka, l'utilisation de `JwtToken` avec des UUID uniques permet de valider l'identité exacte de la session, rendant le système réel plus sûr que sa modélisation simplifiée.

Chapitre 3

Analyse de Sécurité et Big Data

3.1 Pipeline de Données et Persistance

Le système repose sur un flux de données asynchrone. Chaque action critique (tentative de connexion, création de token, verrouillage) génère un `SecurityEvent`[cite : 6]. Ces événements sont capturés par le `ParquetEventWriter` qui les sérialise sur disque.

Avantage du format Parquet

L'utilisation du format Parquet permet de ne charger en mémoire que les colonnes nécessaires (ex : `userId` et `eventType`), optimisant ainsi les ressources lors de l'analyse de millions de logs.

3.2 Algorithme de Détection de Brute-Force

Le `SecurityAnalyzer` charge le dataset et applique une logique d'agrégation via l'API Dataset de Spark[cite : 6].

Listing 3.1 – Logique d'agrégation Spark

```
1 // Filtrage des echecs et groupement par utilisateur
2 val bruteForceUsers = spark.read.parquet(path)
3   .filter($"eventType" === "AUTH_FAILURE")
4   .groupBy($"userId")
5   .count()
6   .filter($"count" >= 3)
7   .as[String]
8   .collect()
```

Cette approche permet d'identifier en quelques secondes les comptes compromis sur une période donnée, fournissant au système une capacité d'audit post-mortem indispensable.

3.3 Détails des Scénarios de Simulation

3.3.1 Scénario 1 : Authentification Nominale

Le client saisit les identifiants corrects. Le système génère un `AUTH_SUCCESS` et crée un token de session.

3.3.2 Scénario 2 : Attaque Brute-force

Le client échoue 3 fois. Le système déclenche le verrouillage du compte et loggue un `ACCOUNT_LOCKED` [1].

3.3.3 Scénario 3 : Replay Attack

Tentative d'accès avec un ancien token. Le `TokenStore` rejette la demande car l'ID du token n'existe plus en mémoire.

3.3.4 Scénario 4 : Credential Stuffing

Plusieurs tentatives sur différents comptes. Spark identifie les anomalies de masse grâce au regroupement par IP/User [2][cite : 6].

Chapitre 4

Validation et Intégration

4.1 Stratégie de Test avec ScalaTest

La validation du projet s'articule autour de deux axes : les tests unitaires pour la logique métier et les tests d'intégration pour le système d'acteurs[cite : 6].

4.1.1 Tests Unitaires (SecurityAnalyzerSpec)

Nous vérifions que l'analyseur Spark identifie correctement les attaquants. Un environnement Spark local est instancié pour simuler des logs de brute-force et valider que l'utilisateur "bob" est bien détecté après 3 échecs simulés [2][cite : 6].

4.1.2 Tests d'Intégration (SecureBankIntegrationSpec)

Ces tests utilisent le `ActorTestKit` d'Akka pour simuler un client réel [1][cite : 6].

Scénario de test critique : Verrouillage

1. Le client tente 3 connexions avec un mauvais mot de passe[cite : 6].
2. L' `AuthServer` incrémente le compteur et finit par envoyer un message `AccountLocked`[cite : 6].
3. Une 4ème tentative avec le **bon** mot de passe est effectuée[cite : 6].
4. Le test valide que l'accès est toujours refusé, prouvant que l'état "Locked" est bien persistant et absorbant [3][cite : 6].

4.2 Couverture Fonctionnelle

Les tests couvrent également la gestion des tokens via le `TokenStore`. Nous vérifions qu'un token expiré ou révoqué entraîne immédiatement une déconnexion du client, conformément au modèle de sécurité défini[cite : 6].

Bibliographie

- [1] Akka Typed Documentation, *Actor Model and FSM*, 2024.
- [2] Apache Spark Documentation, *Parquet Storage and Dataset API*, 2023.
- [3] Murata, T. (1989), *Petri Nets : Properties, Analysis and Applications*.